AWS CLOUD SERVICES — STUDY NOTES

Amazon SQS & SNS

A comprehensive guide to Simple Queue Service and Simple Notification Service — messaging patterns, architecture, configuration, and best practices for cloud-native distributed systems.

DATE	TOPICS	LEVEL
April 7, 2026	SQS · SNS · Fanout · Filtering	Intermediate — Advanced

TABLE OF CONTENTS

01 Introduction

02 Messaging Patterns Overview

03 Amazon SNS Architecture

04 Fanout Pattern (SNS → SQS)

05 Message Filtering

06 Long Polling vs Short Polling

07 Batch Operations

08 Tips & Best Practices

09 Pitfalls & Remedies

10 Review Questions

01 Introduction

Modern distributed applications require **asynchronous communication** to achieve scalability, resilience, and loose coupling. Synchronous point-to-point communication creates tight dependencies — when one service is down or slow, calling services fail or wait indefinitely.

Amazon Simple Queue Service (SQS) and **Amazon Simple Notification Service (SNS)** solve these problems through message queuing and pub/sub patterns. SQS provides reliable message queuing where producers send messages that consumers process asynchronously, decoupling

components and enabling independent scaling. SNS implements publish-subscribe messaging where publishers send messages to topics that fan out to multiple subscribers simultaneously.

REAL-WORLD MOTIVATION

Consider an e-commerce order processing system: when customers place orders, the system must charge payment, update inventory, send confirmation emails, trigger warehouse fulfillment, and update analytics — five independent operations. With SQS/SNS, order placement publishes to an SNS topic, which fans out to five SQS queues processed independently. Order placement returns instantly, operations scale independently, and temporary failures no longer block order acceptance.

WHY DEEP UNDERSTANDING MATTERS

Misconfigured visibility timeouts cause duplicate processing. Missing dead-letter queues lose failed messages permanently. Not using FIFO queues when ordering matters causes data corruption. Improper retry strategies overwhelm systems during failures. Missing message attributes prevent efficient filtering.

02 Messaging Patterns Overview

Why Asynchronous Messaging?

Distributed systems face inherent challenges that messaging patterns address:

Problems with Synchronous Communication

- If Service B is down, Service A fails
- If Service C is slow, everything is slow
- Changes in B or C affect A
- Cascading failures common
- 900 of 1,000 requests/sec fail at peak load
- No buffering mechanism or auto retry
- Service restarts lose in-flight requests

Benefits of Asynchronous Messaging

- Services don't know about each other
- Service B can be down temporarily
- Independent deployment cycles
- Queue buffers load spikes
- Messages persist; automatic retry
- Dead-letter queues for poison messages
- Add subscribers without changing publisher

Core Messaging Patterns

Pattern	Flow	Key Characteristics	Typical Use Cases
Point-to-Point (SQS)	`Producer → Queue → Single Consumer`	Each message processed by one consumer; load balancing; order optional or guaranteed (FIFO)	Job processing, task distribution, work queues, command processing
Publish-Subscribe (SNS)	`Publisher → Topic → Multiple Subscribers`	Each message delivered to all subscribers; fire-and-forget; fanout to multiple endpoints; no persistence	Event notifications, broadcasting updates, multiple system coordination
Fanout (SNS → SQS)	`Publisher → SNS → Multiple SQS → Consumers`	Combines pub/sub and queuing; persistent delivery; independent processing per queue	Event-driven architectures, microservices, parallel processing workflows

03 Amazon SNS Architecture

SNS Core Concepts

Concept	Description
Topic	Named communication channel; publishers send messages to it; fans out to subscribers; no message storage (ephemeral); region-scoped
Publisher	Sends messages to a topic; can be an application, AWS service, or CloudWatch alarm; no knowledge of subscribers; fire-and-forget delivery
Subscriber	Receives messages; types include SQS, Lambda, HTTP/HTTPS, Email, SMS; message delivered to all subscribers with independent processing
Message	Subject (optional, for email), Body (JSON or text, up to 256 KB), Attributes (metadata for filtering), MessageId (unique identifier)

Message Flow

```
Publisher publishes to topic
SNS receives message
SNS fans out to all subscribers (parallel)
├─ SQS Queue
├─ Lambda Function
├─ HTTP/HTTPS endpoint
└─ Email address

Delivery: Best-effort, no guaranteed order (Standard Topic)
```

SNS Subscription Types

1. SQS (Most Common)

Persistent delivery, retry logic, multiple consumers per queue. Best for durable asynchronous processing and fanout to multiple processing systems.

2. Lambda

Immediate processing; no infrastructure management; auto-scales. Ideal for real-time processing, lightweight transformations, and event-driven workflows.

3. HTTP/HTTPS

Integrates with external systems via webhooks. Requires a publicly accessible endpoint that must confirm the subscription and handle message signature verification.

4. Email / Email-JSON

Human notifications and simple alerting. Not suitable for high-volume; delivery not guaranteed; requires manual subscription confirmation.

5. SMS

Mobile notifications with global reach. Per-message charges vary by country. Used for critical alerts, MFA codes, and status updates.

6. Platform Endpoints (Mobile Push)

Supports APNs (Apple), FCM (Google Firebase), and ADM (Amazon). Used for mobile app and device-specific push notifications.

Standard vs FIFO Topics

Attribute	Standard Topic	FIFO Topic
Throughput	Unlimited	300 TPS (3,000 with batching)
Message Ordering	Best-effort	Strict (guaranteed)
Delivery Guarantee	At-least-once (possible duplicates)	Exactly-once
Subscriber Types	All types	SQS FIFO queues only
Naming Requirement	—	Name must end with `.fifo`
Use Cases	Event notifications, high-volume messaging	Event sourcing, ordered state changes, transaction coordination

04 Fanout Pattern (SNS → SQS)

The fanout pattern is one of the most powerful architectures in AWS, combining the broadcast capabilities of SNS with the durability and independent processing of SQS queues.

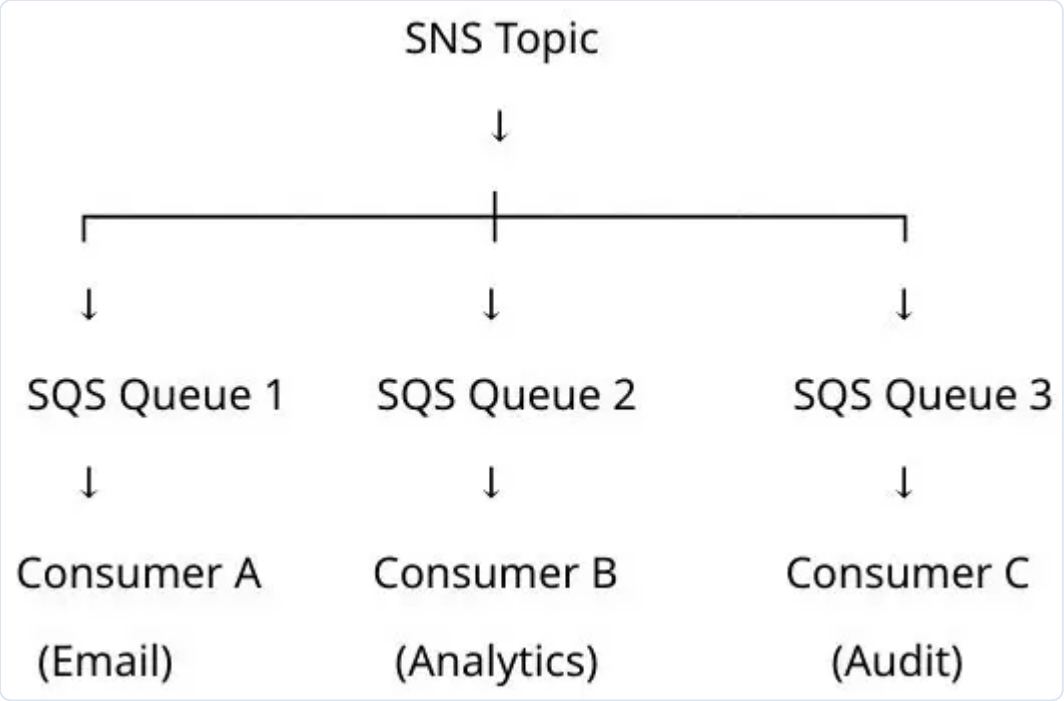

Figure 1: SNS Fanout Architecture — publisher fans out to multiple SQS queues for independent, parallel processing

Benefits

Benefit	Details
Loose Coupling	Publisher doesn't know consumers exist; add/remove consumers without changing publisher
Parallel Processing	All queues receive message simultaneously; independent processing speeds; no blocking between consumers
Durability	SQS stores messages persistently; retry logic per queue; DLQ per consumer; no message loss
Scalability	Each queue scales independently; multiple consumers per queue; auto-scaling based on queue depth

Real-World Example — E-Commerce Order Processing

```
Order Placed → SNS Topic → Fanout
├─ Payment Queue → Process payment
├─ Inventory Queue → Update stock
├─ Email Queue → Send confirmation
├─ Warehouse Queue → Prepare shipment
├─ Analytics Queue → Update metrics
└─ Audit Queue → Log transaction
```

With Fanout (Async)

- Order placement returns instantly
- Each operation independent
- Failure in email doesn't block inventory
- Each system scales separately
- Easy to add new workflows

Without Fanout (Synchronous)

- Slow — sum of all latencies
- Any failure blocks everything
- Complex error handling
- Cannot scale independently
- Rigid coupling between services

Fanout Configuration — Setup Steps

1. **Create SNS Topic** — choose Standard or FIFO; set access policy allowing publish.
2. **Create SQS Queues** (one per consumer) — configure queue settings, set a DLQ for each queue, and configure visibility timeout.

3. **Subscribe Queues to Topic** — create subscription; SNS sends to SQS automatically; set subscription filter policy (optional).
4. **Grant Permissions** — SQS queue policy must allow SNS to send messages; automatic via console, manual via queue policy for CLI/API.

Message Format in SQS

When SNS delivers to SQS, the consumer receives the SNS wrapper. Parse it to extract the actual message body:

```
{
  "Type":              "Notification",
  "MessageId":         "...",
  "TopicArn":          "arn:aws:sns:...",
  "Subject":           "...",
  "Message":           "actual message body",
  "Timestamp":         "...",
  "MessageAttributes": { ... }
}
```

05 Message Filtering

SNS message filtering reduces unnecessary message delivery. Without filtering, all subscribers receive all messages and must filter at the application level — wasting processing, bandwidth, and cost.

Filter Policy Syntax

Type	Example Policy	Matches
String Matching	`{"eventType": ["order_placed", "order_cancelled"]}`	Messages where `eventType` is `order_placed` or `order_cancelled`
Numeric Matching	`{"price": [{"numeric": [">=", 100]}]}`	Messages where `price >= 100`

Prefix Matching	`{"region": [{"prefix": "us-"}]}`	Messages where `region` starts with `us-`
Anything-but	`{"environment": [{"anything-but": ["production"]}]}`	Messages where `environment` is NOT `production`
Exists	`{"userId": [{"exists": true}]}`	Messages that have a `userId` attribute

Complex Example

```
{
  "eventType": ["order_placed"],
  "region":    [{"prefix": "us-"}],
  "amount":    [{"numeric": [">", 100]}],
  "priority":  ["high", "urgent"]
}
/* Matches: eventType=order_placed AND region starts with us-
            AND amount > 100 AND priority=high OR urgent */
```

Real-World Filtering Scenario

Given a message with: `eventType=order_placed`, `region=us-west-2`, `amount=250`, `priority=high`:

Subscriber	Filter Policy	Result
Analytics	`{"eventType": ["order_placed"]}`	Receives message
High-Value Orders	`{"amount": [{"numeric": [">=", 1000]}]}`	Does NOT receive (250 < 1,000)
US Orders	`{"region": [{"prefix": "us-"}]}`	Receives message

BENEFITS OF FILTERING

Reduced message processing, lower costs (fewer SQS messages), less network bandwidth, faster consumer processing, and simpler consumer logic.

06 Long Polling vs Short Polling

Attribute	Short Polling (Default)	Long Polling (Recommended)
Behavior	Returns immediately with messages or empty response	Waits 1–20 seconds for a message to arrive
Empty Responses	Common and frequent	Significantly reduced
API Call Volume	Very high (continuous polling)	Much lower (waits before returning)
Cost Example	Poll every 100ms = 864,000 requests/day (~$10.50/month per consumer)	90–99% reduction in API calls
Configuration	Default; no configuration needed	Set `ReceiveMessageWaitTimeSeconds = 20`
Use When	Immediate response required	Standard queuing workloads (always prefer)

RECOMMENDATION

Always use long polling. Set `WaitTimeSeconds=20` to reduce costs by 90% and improve efficiency. Only use short polling if an immediate response is explicitly required.

07 Batch Operations

Batching significantly improves performance and reduces costs by grouping multiple messages into a single API call.

Individual Operations — 10 Messages

- 10 separate API calls
- Cost: 10 requests charged
- Time: 10 × latency

Batch Operations — 10 Messages

- 1 API call (`send_message_batch`)
- Cost: 1 request charged (90% savings)
- Time: 1 × latency (10× faster)

Batch Limits

Operation	Limit	Notes
SendMessageBatch	Up to 10 messages; 256 KB total payload	Returns success/failure per message
ReceiveMessage	`MaxNumberOfMessages=10`	Returns 1–10 messages; use with long polling
DeleteMessageBatch	Up to 10 messages	Requires `ReceiptHandle` for each
ChangeMessageVisibilityBatch	Up to 10 messages	Efficient for long-running tasks

Cost Comparison — 1 Million Messages/Day

Approach	API Calls	Cost
Without Batching	1,000,000 calls	$0.40/day
With Batching (10/batch)	100,000 calls	$0.04/day — **90% savings**
Annual Savings	—	~$131/year per million daily messages

08 Tips & Best Practices

1. **Always Use Long Polling.** Set `ReceiveMessageWaitTimeSeconds=20` to reduce costs by 90% and improve efficiency.
2. **Implement Idempotency.** Always design consumers to handle duplicate messages safely — use unique IDs and check before processing.
3. **Set Appropriate Visibility Timeout.** Use 2× the average processing time; extend during processing if needed to prevent duplicates.
4. **Use Batch Operations.** Batch send/receive/delete reduces costs by 90% and improves throughput 10×.
5. **Monitor DLQ Continuously.** Set a CloudWatch alarm on DLQ message count — messages in DLQ indicate processing failures requiring attention.

09 Pitfalls & Remedies

1. **Message Duplication (Standard Queue).**
   Problem: Same message processed multiple times causing incorrect state.
   Solution: Implement idempotency using unique message IDs, database constraints, or conditional writes.
2. **Visibility Timeout Too Short.**
   Problem: Timeout expires before processing completes, causing duplicate processing.
   Solution: Set timeout to 2× average processing time; use `ChangeMessageVisibility` for long tasks.
3. **No Dead-Letter Queue.**
   Problem: Poison messages retry forever, blocking the queue and hiding processing failures.
   Solution: Always configure a DLQ with `maxReceiveCount=3–5`; monitor the DLQ for messages.

10 Review Questions

1. What is the maximum SQS message size?

256 KB

2. What is the SQS message retention range?

1 minute to 14 days (default: 4 days)

3. What is the FIFO queue throughput?

300 TPS (3,000 TPS with batching)

4. What is the SNS message delivery guarantee for Standard Topics?

At-least-once (best-effort delivery — possible duplicates)

5. What is the long polling wait time range?

0 to 20 seconds